

CHƯƠNG 9: CÁC THUẬT TOÁN HỌC MÁY

Trong chương này về các thuật toán học máy, chúng ta sẽ tìm hiểu các thuật toán cơ bản của học máy cùng với việc triển khai chúng bằng bộ dữ liệu và thảo luận về ưu nhược điểm của các thuật toán để hiểu rõ hơn và cách sử dụng chúng trong bất kỳ dự án nào hoặc với bất kỳ bộ dữ liệu nào để có được kết quả chính xác hữu ích cho nghiên cứu hoặc học tập.

Thành phần chính

Chúng ta đã định nghĩa một bộ dữ liệu bao gồm âm thanh, hình ảnh và nhãn nhị phân, giúp hiểu được cách chúng ta có thể huấn luyện một mô hình từ các đoạn trích dẫn đến phân loại. Loại bài toán này là một trong nhiều loại bài toán học máy khi chúng ta cố gắng dự đoán một nhãn không xác định cụ thể dựa trên các đầu vào đã biết, với điều kiện có sẵn một bộ dữ liệu các ví dụ đã biết nhãn, được gọi là **học có giám sát**.

Hãy thảo luận về các thành phần chính mà chúng ta cần để đạt được điều đó:

- Dữ liệu
- Mô hình
- Thuật toán

Hồi quy tuyến tính (Linear Regression)

Hồi quy tuyến tính là một phương pháp tuyến tính trong thống kê để mô hình hóa mối quan hệ giữa một hoặc nhiều biến độc lập và một (y) biến phụ thuộc (s). Các mối quan hệ được mô hình hóa trong hồi quy tuyến tính bằng các hàm dự đoán tuyến tính, được ước tính bằng các tham số mô hình chưa biết từ dữ liệu. Hồi quy tuyến tính là một trong những thuật toán phổ biến nhất của Học máy. Đó là vì tính đơn giản tương đối và các thuộc tính phổ biến của nó. Nó có hai loại là:

- **Hồi quy tuyến tính đơn giản (Simple Linear Regression):** Nếu bạn chỉ làm việc với một biến độc lập, hồi quy tuyến tính được gọi là đơn giản. Nó được biểu thị bằng công thức $f(x) = mx + b$.

Chúng ta sẽ sử dụng hai hàm như chi phí (cost) và gradient để tính toán và tối ưu hóa mô hình của mình với dữ liệu chúng ta cung cấp.

Hàm chi phí (Cost Function): Chúng ta tính toán độ chính xác bằng hàm chi phí lỗi bình phương trung bình (MSE) của thuật toán hồi quy tuyến tính. MSE tính toán khoảng cách bình phương trung bình giữa hiệu suất dự kiến và hiệu suất thực tế. Công thức của nó là:

$$Error(m, b) = \frac{1}{N} \sum_{i=1}^N (\text{Đầu ra thực tế} - \text{Đầu ra dự đoán})$$

MSE rất đơn giản và có thể dễ dàng lập trình bằng Python như:

```
def cost_function(m, b, x, y):
    totalError = 0
    for i in range(0, len(x)):
        totalError += (y[i] - (m*x[i] + b))**2
    return totalError / float(len(x))
```

Tối ưu hóa: Chúng ta sẽ sử dụng gradient descent (giảm độ dốc) để tìm các hệ số giúp giảm thiểu hàm lỗi. Gradient descent là một thuật toán tối ưu hóa, lặp đi lặp lại các bước đi về phía cực tiểu cục bộ của hàm chi phí.

Để tìm đường xuống “đáy” (cực tiểu), chúng ta phải lấy đạo hàm của hàm lỗi theo m và b. Sau đó, chúng ta thực hiện một bước theo hướng âm (ngược hướng gradient).

Công thức Gradient Descent tổng quát

$$\Theta_j = \Theta_j - \alpha \frac{\partial}{\partial \Theta_j} J(\Theta_0 - \Theta_1)$$

Gradient Descent cho Hồi quy tuyến tính đơn giản.

$$\frac{\partial}{\partial m} = \frac{2}{N} \sum_{i=1}^N -x_i (y_i - (mx_i + b))$$

$$\frac{\partial}{\partial b} = \frac{2}{N} \sum_{i=1}^N - (y_i - (mx_i + b))$$

Việc triển khai gradient descent phức tạp hơn một chút, nhưng cũng có thể dễ dàng thực hiện bằng Python thuần túy:

```
def gradient_descent(b, m, x, y, learning_rate, num_iterations):
    N = float(len(x))
    for j in range(num_iterations): # lặp lại cho num_iterations
        b_gradient = 0
        m_gradient = 0
        for i in range(0, len(x)):
            b_gradient += -(2/N) * (y[i] - ((m*x[i]) + b))
            m_gradient += -(2/N) * x[i] * (y[i] - ((m*x[i]) + b))
        b -= (learning_rate * b_gradient)
        m -= (learning_rate * m_gradient)
        if j % 50 == 0:
            print('error:', cost_function(m, b, x, y))
    return [b, m]
```

Triển khai Hồi quy tuyến tính

Chúng ta phải xây dựng một bộ dữ liệu và xác định một số biến ban đầu để chạy mô hình hồi quy tuyến tính của mình. Để hiển thị dữ liệu ngẫu nhiên đơn giản, chúng ta sử dụng NumPy và matplotlib.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 100, 50)
delta = np.random.uniform(-10, 10, x.size)
y = 0.6*x + 5 + delta

plt.scatter(x, y)
plt.show()
```

Đầu ra: như trong Hình 9.1

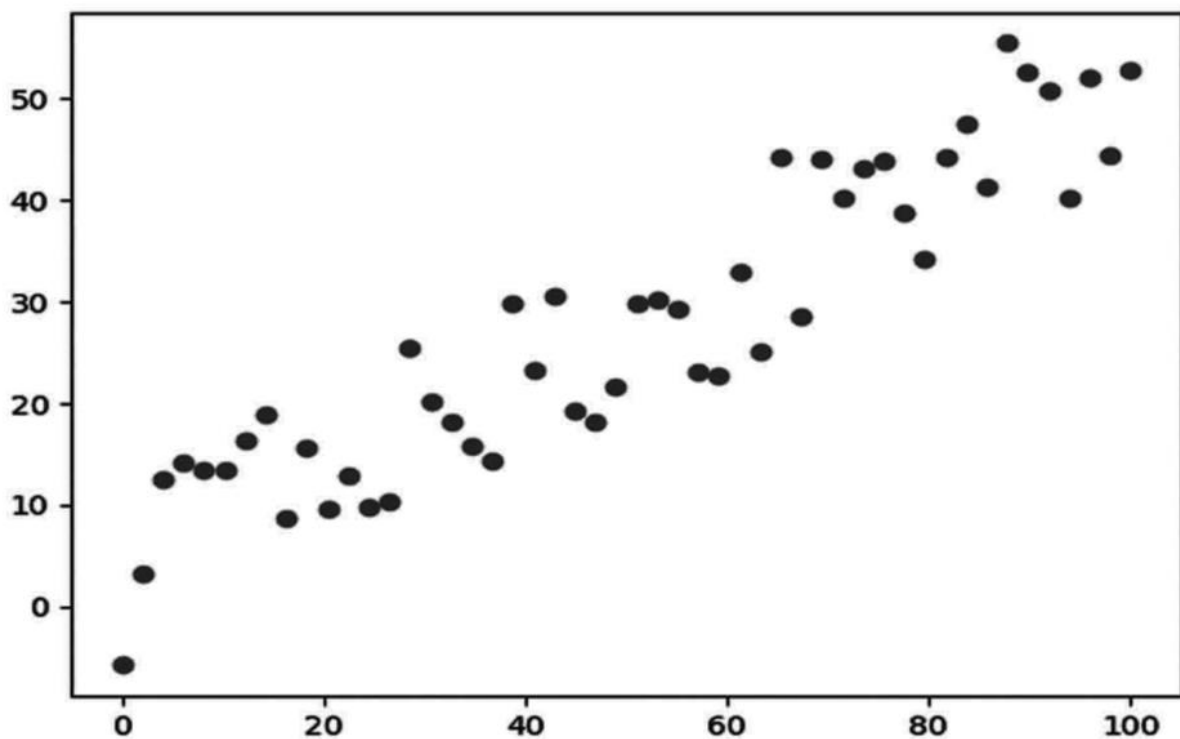


Fig. 9.1 Program Output

Bây giờ chúng ta đã có dữ liệu, chúng ta sẵn sàng sử dụng chức năng trên để huấn luyện mô hình của mình:

```
# defining some variables
learning_rate = 0.0001
```

```
initial_b = 0
initial_m = 0
num_iterations = 100

print('Initial error:', cost_function(initial_m, initial_b, x, y))
[b, m] = gradient_descent(initial_b, initial_m, x, y, learning_rate, num_iterations)
print('b:', b)
print('m:', m)
print('error:', cost_function(m, b, x, y))
plt.show()
```

Đầu ra:

```
Initial error: 1576.099063465165
error: 205.2155773301355
error: 41.73021903795362
b: 0.03851474971994741
m: 0.6744456473643745
error: 41.69053444198719
```

Bây giờ chúng ta có thể sử dụng mô hình của mình như một công cụ dự đoán, vì chúng ta đã huấn luyện nó. Chúng ta sẽ chỉ dự đoán dữ liệu huấn luyện trong ví dụ đơn giản này và sử dụng kết quả để vẽ đường thẳng phù hợp nhất bằng matplotlib.

```
predictions = [(m*x[i]) + b for i in range(len(x))]
plt.scatter(x, y)
plt.plot(x, predictions, color='r')
plt.show()
```

Đầu ra: như trong Hình 9.2

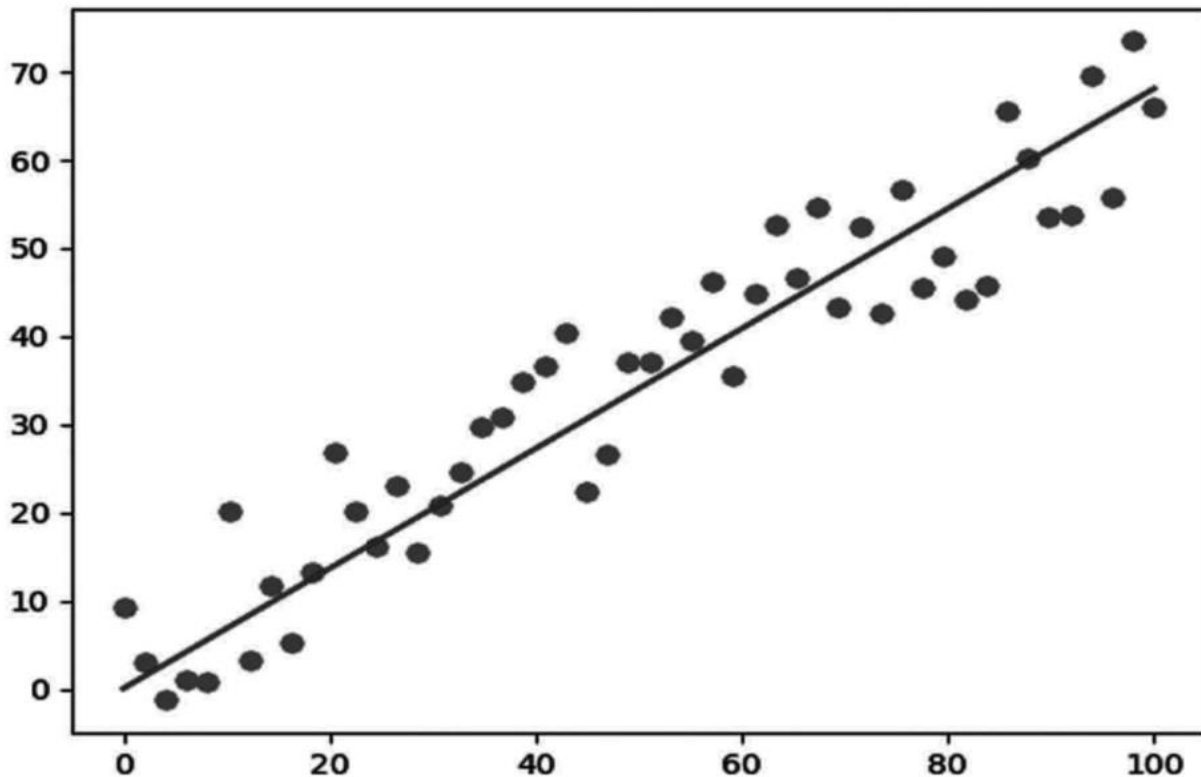


Fig. 9.2 Program Output

- **Hồi quy tuyến tính đa biến (Multiple Linear Regression):** Khi bạn xử lý ít nhất hai biến hoạt động độc lập, hồi quy tuyến tính được gọi là đa biến. Mỗi biến (còn được gọi là đặc trưng) được nhân với một trọng số mà thuật toán hồi quy tuyến tính của chúng ta đã học được. Nó được biểu thị bằng công thức:

$$f(x) = b + W_1X_1 + W_2X_2 + \dots + W_nX_n = b + \sum_{i=0}^n W_i X_i$$

Chúng ta sẽ sử dụng hai hàm như chi phí và gradient để tính toán và tối ưu hóa mô hình của mình với dữ liệu chúng ta cung cấp.

Hàm chi phí: Chúng ta sẽ sử dụng lỗi bình phương trung bình làm hàm mất mát, giống như chúng ta đã làm với Hồi quy tuyến tính đơn giản. Sự khác biệt duy nhất là hiệu suất dự đoán của chúng ta bây giờ đến từ một đặc trưng khác. Vì bây giờ có thể sử dụng NumPy để lập trình hàm chi phí của chúng ta với nhiều đặc trưng và trọng số.

Hồi quy Logistic (Logistic Regression)

Đây là một thuật toán phân loại, được sử dụng khi các loại của biến phản hồi thay đổi. Ý tưởng của hồi quy là tạo ra một liên kết giữa các đặc trưng và khả năng xảy ra của một kết quả cụ thể.

Ví dụ. Biến câu trả lời có hai giá trị, “đỗ” và “trượt” khi chúng ta cần dự đoán liệu một sinh viên đỗ hay trượt kỳ thi nếu số giờ học được coi là yếu tố.

Loại bài toán này được gọi là **hồi quy logistic nhị thức (binomial logistical regression)**, trong đó biến câu trả lời có hai giá trị 0 và 1, hoặc đỗ và trượt, hoặc đúng và sai.

Hồi quy Logistic đa thức (Multinomial Logistic Regression) thảo luận về các tình huống mà biến phản hồi có thể có ba hoặc nhiều giá trị tiềm năng.

Tại sao dùng Hồi quy Logistic mà không dùng Hồi quy tuyến tính?

Đặt ‘x’ là một hàm nào đó với phân loại nhị phân, và đặt y^s là đầu ra có thể là 0 hoặc 1. Xét đầu vào, xác suất để đầu ra là 1 là: $P(y = 1/x)$.

Chúng ta có thể nói rằng nếu chúng ta ước tính xác suất bằng hồi quy tuyến tính: $p(X) = \beta_0 + \beta_1 X$ trong đó, $p(x) = p(y = 1/x)$

Mô hình hồi quy tuyến tính có thể tạo ra xác suất dự đoán là bất kỳ số nào nằm trong khoảng từ âm đến dương, trong khi kết quả (xác suất) chỉ có thể nằm trong khoảng $0 < P(x) < 1$ như trong Hình 9.3.

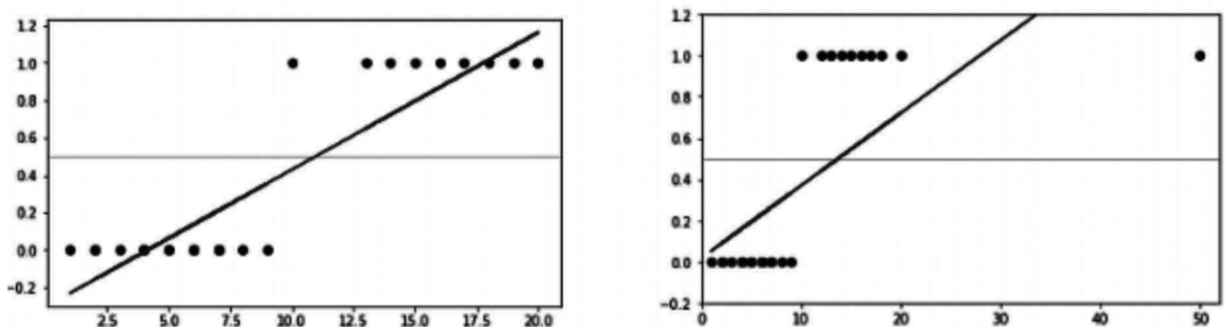


Fig. 9.3 Logistic Regression

Hồi quy tuyến tính cũng bị ảnh hưởng lớn bởi các điểm ngoại lệ (outliers). Để tránh vấn đề này, hàm log-odds hoặc hàm logistic được sử dụng.

Hàm Logistic/Logit

Hồi quy logistic có thể được biểu thị là:

$$\log \left(\frac{p(X)}{1 - p(X)} \right) = \beta_0 + \beta_1 X$$

Hàm logit hoặc log-odds nằm ở vế trái, và $p(x)/(1 - p(x))$ được gọi là “odds” (tỷ lệ chênh).

Tỷ lệ chênh biểu thị tỷ lệ giữa xác suất thành công và xác suất thất bại. Do đó, một tổ hợp tuyến tính của các đầu vào với $\log(\text{odds})$ trong Hồi quy Logistic được ánh xạ - đầu ra bằng 1.

Nếu đảo ngược hàm trên, chúng ta sẽ có:

$$p(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$$

Phương trình này được gọi là **hàm Sigmoid**, là một đường cong có hình chữ S. Nó cũng cung cấp một giá trị xác suất $0 < p < 1$.

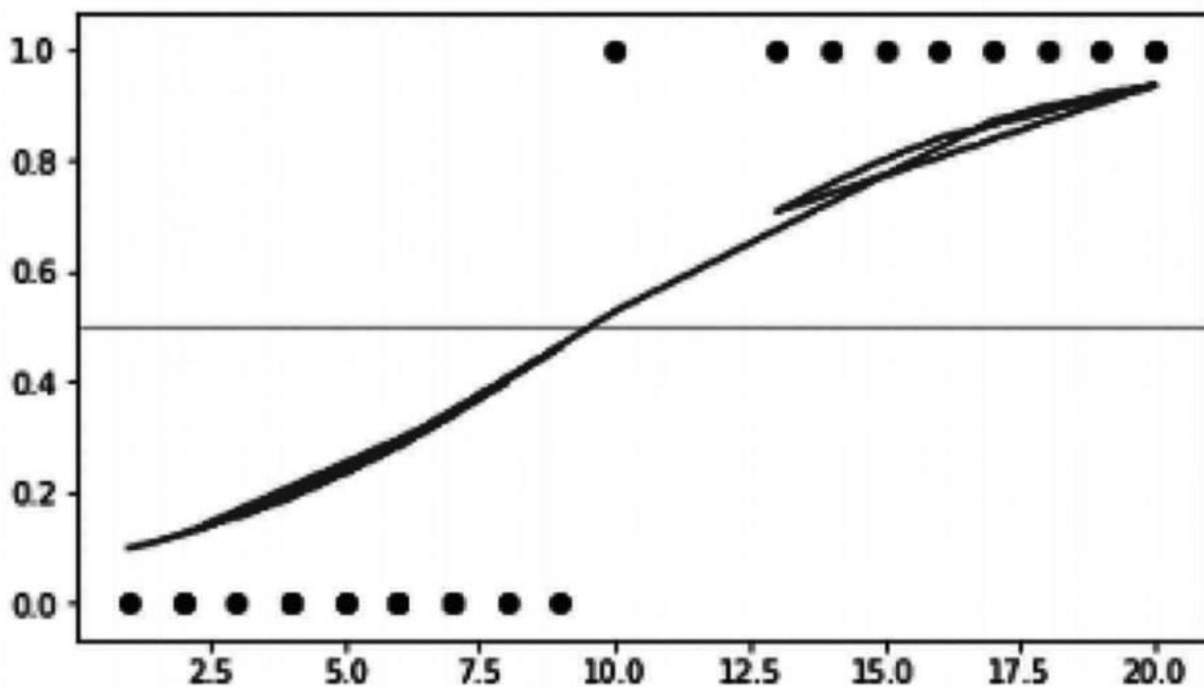


Fig. 9.4 Sigmoid Function

Ước tính các hệ số hồi quy

Chúng ta sử dụng **Ước tính Hợp lý Tối đa (Maximum Likelihood Estimation - MLE)**, không giống như hồi quy tuyến tính sử dụng Bình phương Tối thiểu Thông thường (Ordinary Least Square) để ước tính tham số.

Có thể có vô số tập hợp các hệ số hồi quy. MLE tìm ra chuỗi các hệ số hồi quy mang lại khả năng tối đa để thu được dữ liệu mà chúng ta đã quan sát.

Nếu chúng ta có kết quả nhị phân, nó là α nếu kết quả tốt và $1 - \alpha$ trong trường hợp ngược lại. Do đó, chúng ta có hàm xác suất:

$$L(\beta; y) = \prod_{i=1}^N \left(\frac{\pi_i}{1 - \pi_i} \right)^{y_i} (1 - \pi_i)$$

Để đánh giá giá trị tham số, hàm log của xác suất (log-likelihood) được sử dụng vì nó không làm thay đổi các thuộc tính của hàm. Các giá trị tham số giúp tối đa hóa log-likelihood được tính toán bằng các kỹ thuật lặp như phương pháp Newton.

Hiệu suất của Mô hình Hồi quy Logistic

Độ lệch (Deviance) được sử dụng thay vì tổng bình phương để kiểm tra đầu ra của mô hình hồi quy logistic như trong bảng 9.1.

- **Độ lệch Null (Null Deviance):** thể hiện phản ứng dự kiến của mô hình chỉ có hệ số chặn (interception).
- **Độ lệch Hệ thống (System Deviance):** chỉ ra phản ứng dự kiến của hệ thống khi bổ sung các biến độc lập. Nếu phương sai của mô hình nhỏ hơn đáng kể so với độ lệch null, có thể suy ra rằng tham số hoặc tập hợp các tham số đó có ý nghĩa.

Ma trận nhầm lẫn (Confusion Matrix) là một cách khác để đánh giá độ chính xác của mô hình.

Bảng 9.1 Hiệu suất mô hình Hồi quy Logistic

	P' (Dự đoán)	n' (Dự đoán)
P (Thực tế)	True Positive	False Negative
n (Thực tế)	False Positive	True Negative

Độ chính xác (Accuracy) của mô hình được định nghĩa bởi:

$$\frac{TruePositives + TrueNegatives}{TruePositive + TrueNegatives + FalsePositives + FalseNegatives}$$

Hồi quy Logistic đa lớp (Multi-class Logistic Regression)

Đằng sau hồi quy đa lớp và nhị thức là cùng một trực giác cơ bản. Tuy nhiên, chúng ta theo đuổi cách tiếp cận **một-so-với-tất-cả (one-vs-rest)** cho các vấn đề đa lớp.

Ví dụ. Chúng ta xử lý Bài toán Đa lớp nếu phải dự đoán thời tiết là “nắng”, “mưa” hay “có gió”. Chúng ta chuyển điều này thành ba câu hỏi phân loại nhị thức, cụ thể là: (1) nắng so với không nắng, (2) mưa so với không mưa, và (3) có gió so với không có gió. Cả ba phân loại này được thực hiện độc lập. Giải pháp (lớp dự đoán cuối cùng) là lớp có giá trị xác suất lớn nhất so với các lớp khác.

Triển khai Hồi quy Logistic

Chúng ta sẽ triển khai hồi quy logistic một cách đơn giản bằng thư viện Scikit-learn. Chúng ta sẽ sử dụng **Bộ dữ liệu Iris**.

Bộ dữ liệu Iris có lẽ là một trong những bộ dữ liệu học máy đơn giản nhất. Nó mô tả các loài hoa iris (hoa diên vĩ) ở dạng số, ghi lại các phép đo chiều dài/chiều rộng của đài hoa (sepal) và cánh hoa (petal). Chúng ta sẽ cố gắng dự đoán các loài hoa dựa trên các phép đo này.

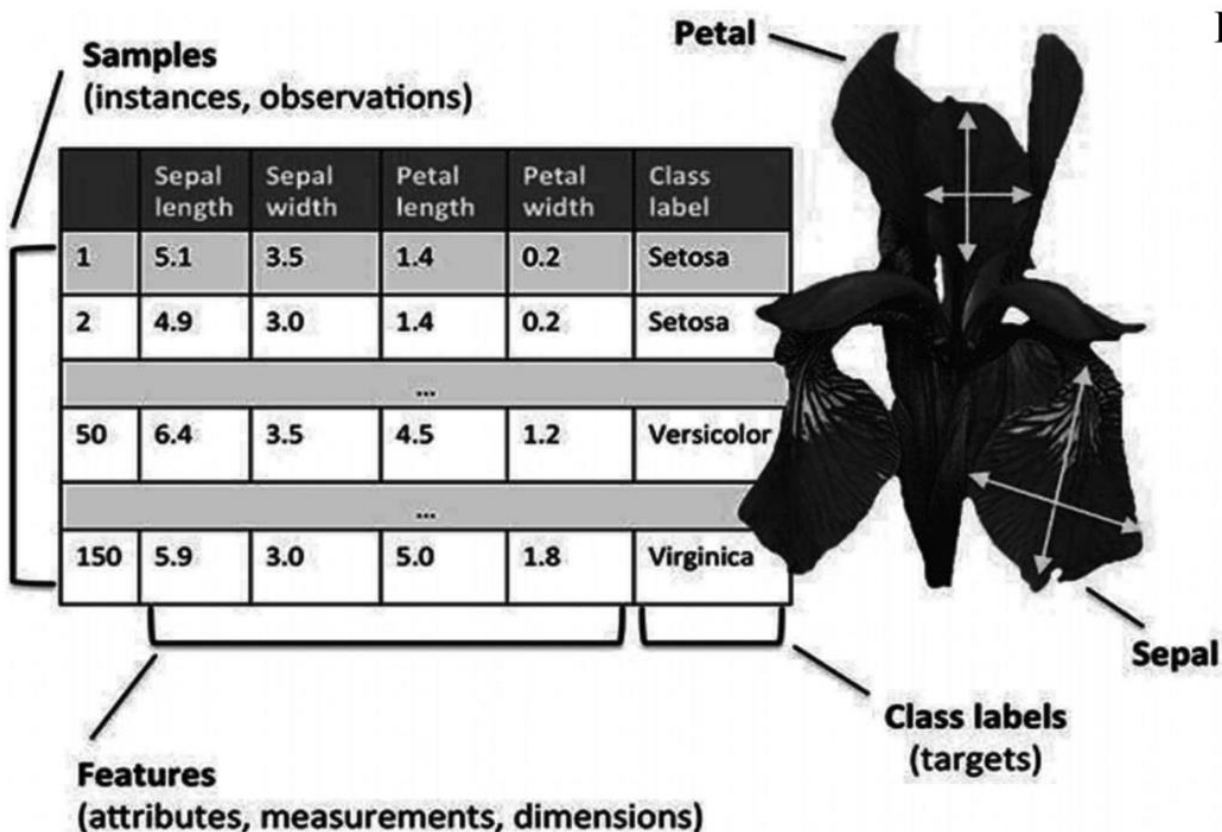


Fig. 9.5 Dataset Description

Bộ dữ liệu này có các cột: sepal length cm, sepal width cm, petal length cm, petal width cm, và species. Bốn cột đầu tiên là đặc trưng và cột cuối cùng là mục tiêu.

Bạn có thể chạy mã này trong bất kỳ môi trường Python nào.

Bước 1: chúng ta sẽ nhập tất cả các thư viện cần thiết

```
# import dependencies
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt
# Note: The symbol # represent the comment for the program
```

Bước 2: tải dữ liệu

```
from sklearn import datasets
data = datasets.load_iris()
# here as we have imported the data through sckit's package,
# we will get the data in form of arrays,
# so here our labes is stored in
data.data
```

Đầu ra:

```
array([[5.1, 3.5, 1.4, 0.2], [4.9, 3. , 1.4, 0.2], [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2], [5. , 3.6, 1.4, 0.2], [5.4, 3.9, 1.7, 0.4],
       [4.6, 3.4, 1.4, 0.3], [5. , 3.4, 1.5, 0.2], [4.4, 2.9, 1.4, 0.2],
       [4.9, 3.1, 1.5, 0.1], [5.4, 3.7, 1.5, 0.2], [4.8, 3.4, 1.6, 0.2],
       [4.8, 3. , 1.4, 0.1], [4.3, 3. , 1.1, 0.1], [5.8, 4. , 1.2, 0.2],
       [5.7, 4.4, 1.5, 0.4], [5.4, 3.9, 1.3, 0.4], [5.1, 3.5, 1.4, 0.3],
       [5.7, 3.8, 1.7, 0.3],...so on...
```

```
# Targets for labels
data.target
```

Đầu ra:

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

Bước 3: Bây giờ chúng ta sẽ chia dữ liệu

```
# we will divide the dataset into data and target for the model
X = data.data[:, :2] # X is the features in our dataset
y = (data.target != 0) * 1 # y is the Labels in our dataset
```

Sau khi tách, chúng ta sẽ trực quan hóa nó trên biểu đồ bằng matplotlib:

```
# here we have only taken the x line as SepalLengthCm and y line as SepalWidthCm
plt.figure(figsize=(10, 6))
plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1], color='g', label='0')
```

```
plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], color='r', label='1')
plt.legend()
```

Đầu ra: Như trong Hình 9.6

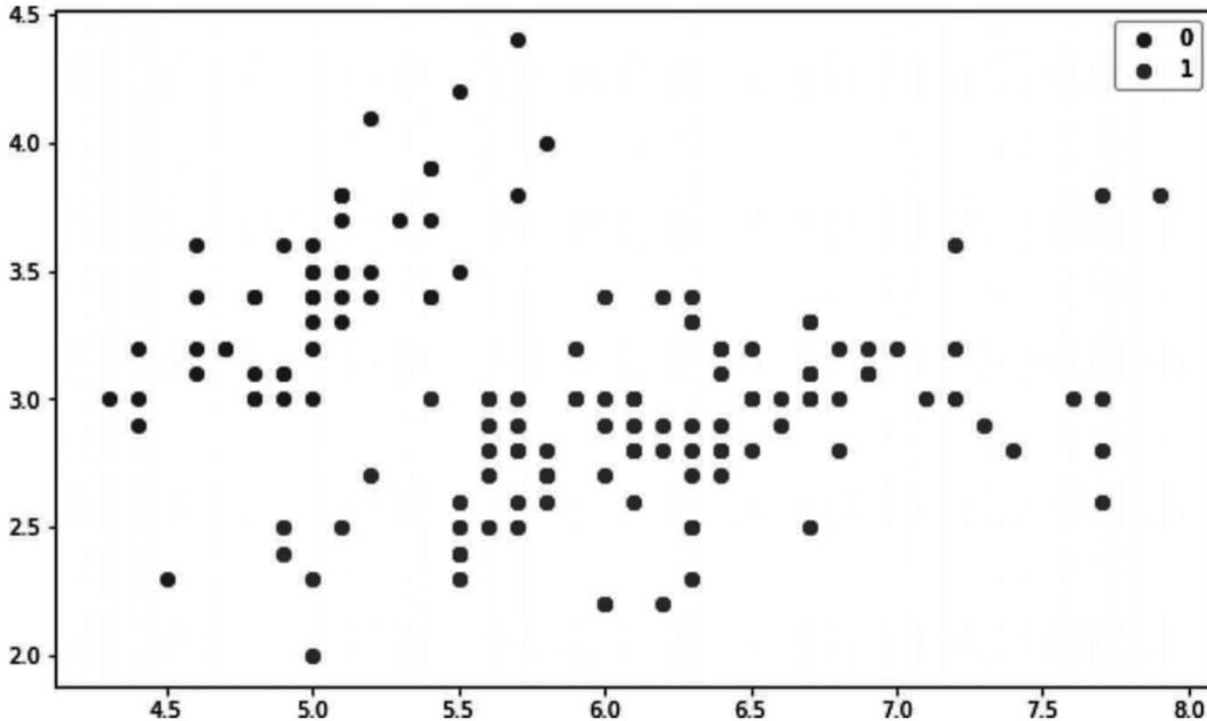


Fig. 9.6 Program Output

Bước 4: Sử dụng mô hình hồi quy logistic của scikit-learn để huấn luyện và dự đoán.

```
# import library with model
from sklearn.linear_model import LogisticRegression
# loading the model which c as parameter which can be given to
# strengthen the model with predefined values like learning rate, iteration etc.,
model = LogisticRegression(C=1e20)
# now we will train the model
model.fit(X, y)
```

Đầu ra:

```
LogisticRegression(C=1e+20, class_weight=None, dual=False,
    fit_intercept=True, intercept_scaling=1, l1_ratio=None,
    max_iter=100, multi_class='auto', n_jobs=None, penalty='l2',
    random_state=None, solver='lbfgs', tol=0.0001, verbose=0,
    warm_start=False)
```

```
# Prediction from model
preds = model.predict(X)
(preds == y).mean()
```

Đầu ra:

1.0

Ưu điểm:

- Đơn giản và hiệu quả.
- Phương sai thấp.
- Cung cấp điểm xác suất cho các quan sát.
- Cũng có thể được sử dụng để trích xuất đặc trưng.

Nhược điểm:

- Không xử lý tốt một số đặc trưng/thuộc tính dạng danh mục.
- Cần biến đổi các đặc trưng phi tuyến tính.
- Không đủ mạnh mẽ để xử lý các mối quan hệ phức tạp hơn.

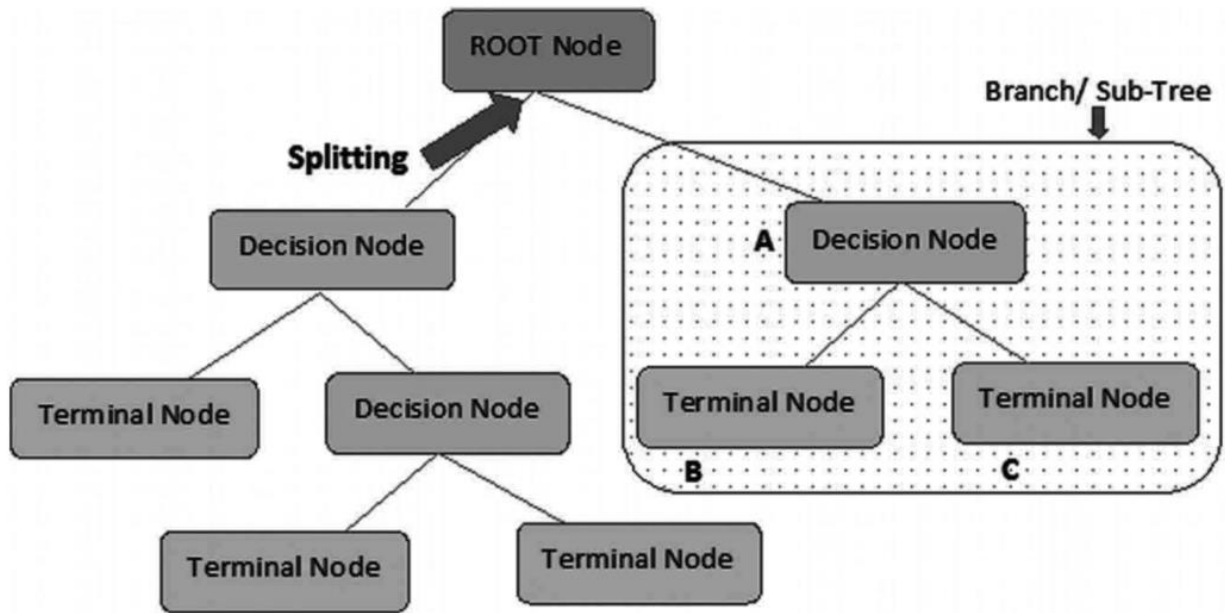
Cây quyết định (Decision Tree)

Thuật toán cây quyết định thuộc danh mục học có giám sát. Cây quyết định có thể được sử dụng để biểu diễn các quyết định và việc ra quyết định một cách trực quan và rõ ràng. Như tên gọi, thuật toán này sử dụng một mẫu quyết định giống như cây. Mặc dù là một kỹ thuật khai thác dữ liệu thường được sử dụng cho một chiến lược để đạt được mục tiêu nhất định, nó cũng thường được sử dụng rộng rãi trong học máy.

Cây quyết định được thiết kế bằng cách sử dụng các thuật toán nhận dạng các cách để chia một tập hợp dữ liệu dựa trên các tiêu chí khác nhau. **Cây phân loại (Classification trees)** là các mô hình cây trong đó biến mục tiêu sẽ nhận một tập hợp các giá trị riêng biệt. **Cây hồi quy (Regression trees)** là cây quyết định trong đó biến mục tiêu có thể nhận các giá trị liên tục (thường là số thực). Thuật ngữ chung cho loại này là **Cây Phân loại và Hồi quy (CART)**.

Thuật ngữ quan trọng trong Cây quyết định

Hãy thảo luận về các thuật ngữ cơ bản được sử dụng trong cây quyết định có thể thấy trong Hình 9.7.



Note:- A is parent node of B and C.

Fig. 9.7 Decision tree representation

Ghi chú: - A là nút cha của B và C.

- **Nút gốc (Root node):** Đại diện cho toàn bộ tập hợp hoặc mẫu và được chia nhỏ thành hai hoặc nhiều mẫu đồng nhất hơn.
- **Phân chia (Splitting):** Đây là cơ chế một nút được chia thành hai hoặc nhiều nút con.
- **Nút quyết định (Decision Node):** Nếu một nút con chia thành các nút con khác, nó được gọi là nút quyết định.
- **Nút cuối/Lá (Terminal Node/Leaf):** Các nút không có con (không phân chia thêm) được gọi là nút Lá và nút Cuối.
- **Cắt tỉa (Pruning):** khi chúng ta giảm kích thước của cây quyết định bằng cách loại bỏ các nút (ngược lại với phân chia).
- **Nhánh/Cây con (Branch/Sub-Tree):** Được gọi là một phần của cây quyết định.
- **Nút Cha và Nút Con (Parent and Child Node):** Một nút được chia thành các nút con được gọi là nút cha của các nút con, trong đó nút con là con của một nút cha.

Phương pháp xây dựng Cây quyết định

Thách thức lớn nhất trong Cây quyết định là chọn thuộc tính cho nút gốc và cho mỗi cấp độ. Phương pháp này được gọi là **lựa chọn thuộc tính**. Chúng ta có hai thước đo lựa chọn thuộc tính phổ biến:

- **Lợi ích thông tin (Information Gain):** Khi chúng ta sử dụng một nút trong cây quyết định, các mẫu huấn luyện được chia thành các tập con có entropy nhỏ hơn.

Sự thay đổi entropy này được xác định bằng lợi ích thông tin. Chúng ta đặt nhiều loại câu hỏi khác nhau ở mỗi nút của cây trong quá trình ra quyết định. Trên cơ sở câu hỏi được đặt ra, chúng ta sẽ đo lường lợi ích dữ liệu.

- **Định nghĩa:** Giả sử S là một tập hợp các mẫu, A là một thuộc tính, S_v là tập con của S với $A = v$, và $\text{Values}(A)$ là tập hợp tất cả các giá trị có thể có của A , thì:

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

- **Entropy:** Entropy là yếu tố không chắc chắn (hoặc độ hỗn loạn) của một biến ngẫu nhiên và nó mô tả sự không tinh khiết của các mẫu tùy ý. Entropy càng lớn, giá trị của thông tin (cần để giảm sự không chắc chắn) càng lớn.
- **Chỉ số Gini (Gini Index):** Chỉ số Gini là thước đo tần suất một phân tử được chọn ngẫu nhiên từ tập hợp sẽ bị phân loại sai. Điều này có nghĩa là bạn sẽ chọn một thuộc tính có chỉ số Gini thấp hơn (ít hỗn loạn hơn). Sklearn mặc định sử dụng chỉ số “Gini”.
 - Công thức dưới đây được đưa ra để tính Chỉ số Gini:

$$\text{GiniIndex} = 1 - \sum_j p_{j2}$$

(trong đó p_j là tỷ lệ của các mẫu thuộc lớp j)

Triển khai Cây quyết định

Hãy xem cách triển khai phân loại cây quyết định trong Python.

Bước 1: chúng ta nhập các thư viện.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
from sklearn.tree import export_graphviz
from sklearn.externals.six import StringIO
from IPython.display import Image
from pydot import graph_from_dot_data
import pandas as pd
import numpy as np
```

Bước 2: Tải bộ dữ liệu

```
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Categorical.from_codes(iris.target, iris.target_names)
X.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Bước 3: Phân chia dữ liệu

```
y = pd.get_dummies(y)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=1)
```

Bước 4: Sử dụng bộ phân loại cây quyết định để phân loại hoa

```
dt = DecisionTreeClassifier()
dt.fit(X_train, y_train)
```

Đầu ra:

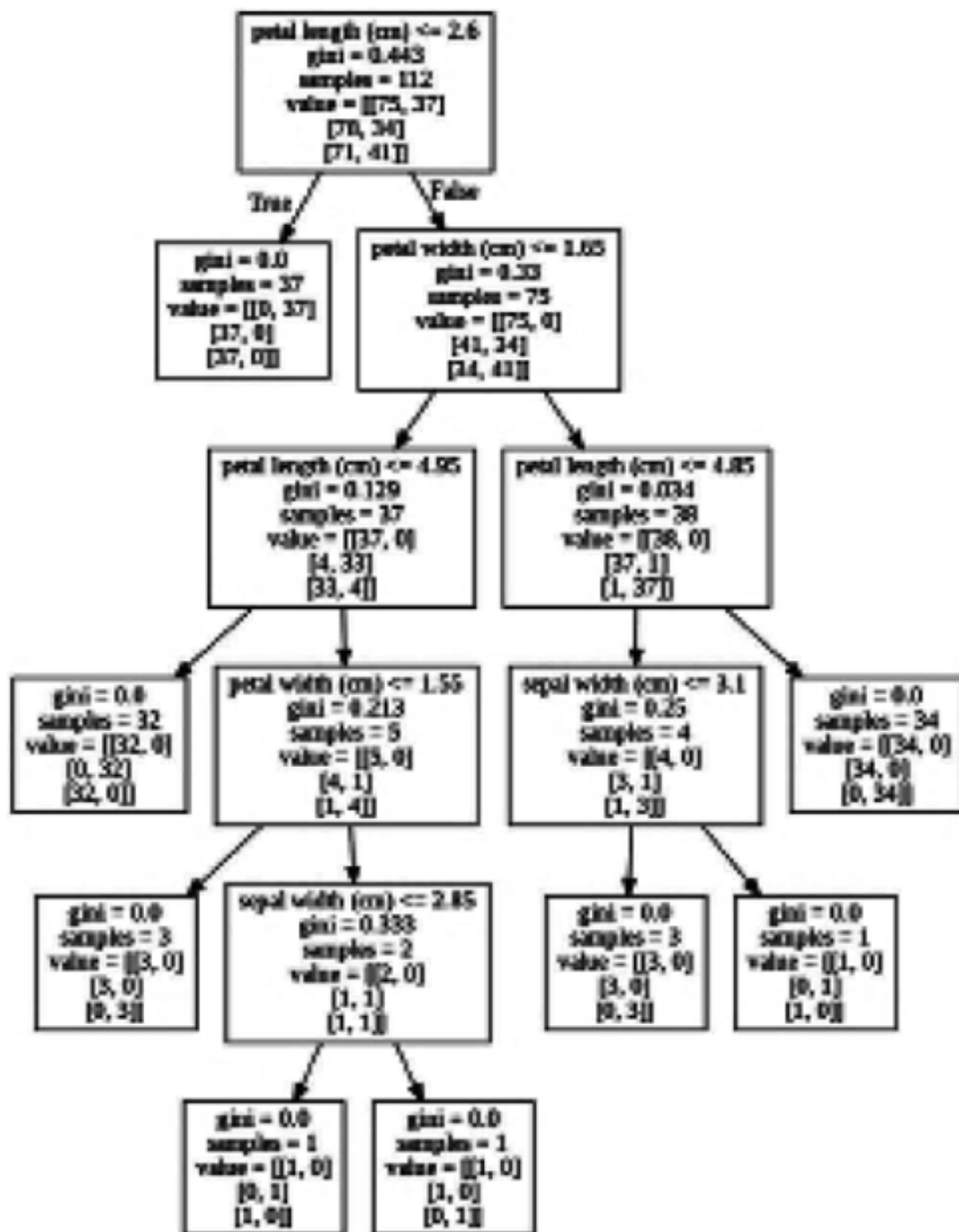
```
DecisionTreeClassifier(ccp_alpha=0.0, class_weight=None, criterion='gini',
                        max_depth=None, max_features=None, max_leaf_nodes=None,
                        min_impurity_decrease=0.0, min_impurity_split=None,
                        min_samples_leaf=1, min_samples_split=2,
                        min_weight_fraction_leaf=0.0, presort='deprecated',
                        random_state=None, splitter='best')
```

Bước 5: Biên dịch và trực quan hóa cây quyết định

```
dot_data = StringIO()
export_graphviz(dt, out_file=dot_data,
                feature_names=iris.feature_names)
(graph,) = graph_from_dot_data(dot_data.getvalue())
Image(graph.create_png())
```

Lưu ý cách nó bao gồm độ không tinh khiết Gini, tổng số mẫu, tiêu chí phân loại và số lượng mẫu bên trái/phải.

Đầu ra:



Bước 6: Dự đoán bằng ma trận nhầm lẫn

```

y_pred = dt.predict(X_test)
species = np.array(y_test).argmax(axis=1)
predictions = np.array(y_pred).argmax(axis=1)
confusion_matrix(species, predictions)

```

Đầu ra:


```
array([[13, 0, 0],  
       [ 0, 15, 1],  
       [ 0, 0, 9]])
```

Chúng ta có thể thấy rằng cây quyết định của chúng ta đã phân loại chính xác 37/38 cây (13+15+9=37).

Máy Vector Hỗ trợ (Support Vector Machine - SVM)

Máy Vector Hỗ trợ (SVM) là một phương pháp học máy dùng cho phân loại và phân tích hồi quy. Nó dựa trên các mô hình học có giám sát và huấn luyện bằng thuật toán. Lượng lớn dữ liệu được phân tích để tìm ra các xu hướng.

Một SVM tạo ra hai đường song song để phân vùng song song. Đối với mỗi danh mục dữ liệu, nó sử dụng hầu hết mọi thuộc tính trong một không gian nhiều chiều. Nó phân chia không gian thành các phân vùng phẳng và tuyến tính trong một lần duy nhất. Mục tiêu là chia 2 nhóm với khoảng cách rõ ràng nhất có thể. Điều này được thực hiện bằng một mặt phẳng gọi là **siêu phẳng (hyperplane)**.

Một SVM tạo ra siêu phẳng với **lề (margin)** lớn nhất để chia dữ liệu thành các lớp trong một không gian nhiều chiều.

Khoảng cách giữa hai lớp là khoảng cách lớn nhất giữa các điểm dữ liệu gần nhất của các lớp đó. Lề càng lớn, lỗi tổng quát hóa phân loại của bộ phân loại càng thấp (như trong Hình 9.8).

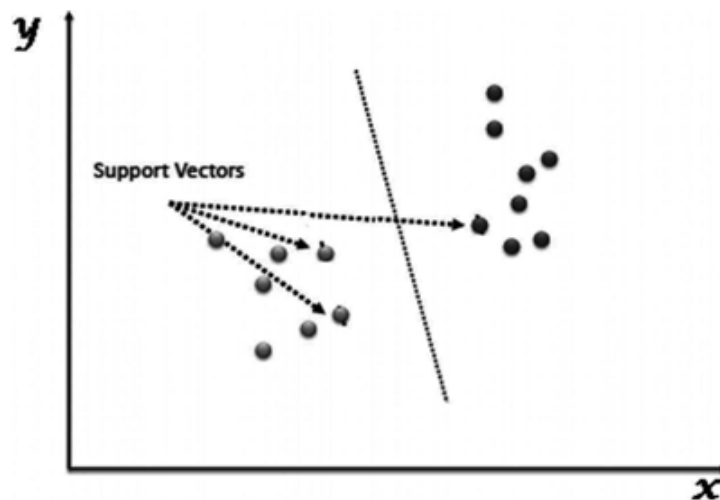


Fig. 9.8 SVM Representation

Cách thức hoạt động?

Hãy xem xét hai điều kiện để nắm bắt thuật toán SVM:

- **Trường hợp tách biệt (Separate case):** Vô số ranh giới có thể được dùng để chia hai nhóm.

- **Trường hợp không tách biệt (Non-separable case):** Không có sự phân chia rõ ràng giữa hai nhóm, mà có sự chồng chéo.

Triển khai SVM

Bước 1: Nhập các thư viện

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
```

Bước 2: Nhập và sử dụng bộ dữ liệu từ Google Drive

```
from google.colab import drive
drive.mount('/gdrive')
cd /gdrive
```

Đầu ra:

Go to this URL in a browser:

https://accounts.google.com/o/oauth2/auth?client_id=94.....

Enter your authorization code:

Mounted at /gdrive

Đây là mã để gắn (mount) drive của bạn với Google Research Collaboratory, nó sẽ yêu cầu bạn xác thực bằng tài khoản Gmail của mình bằng cách làm theo các bước đơn giản.

```
# nếu bạn đang sử dụng trình biên tập python, chỉ cần cung cấp đường dẫn tệp
# nơi bạn đã lưu trữ nó trên hệ thống của mình
# Nhập bộ dữ liệu bằng thư viện Seaborn
iris = pd.read_csv('/gdrive/My Drive/Colab Notebooks/IRIS.csv') #cung cấp đường dẫn
cho bộ dữ liệu
# Kiểm tra bộ dữ liệu
iris.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Bước 3: Trực quan hóa sự tương đồng trong bộ dữ liệu bằng pair plots

Tạo một pairplot để trực quan hóa sự tương đồng và đặc biệt là

sự khác biệt giữa các loài

sns.pairplot(data=iris, hue='species', palette='Set2')

Đầu ra: như trong Hình 9.9

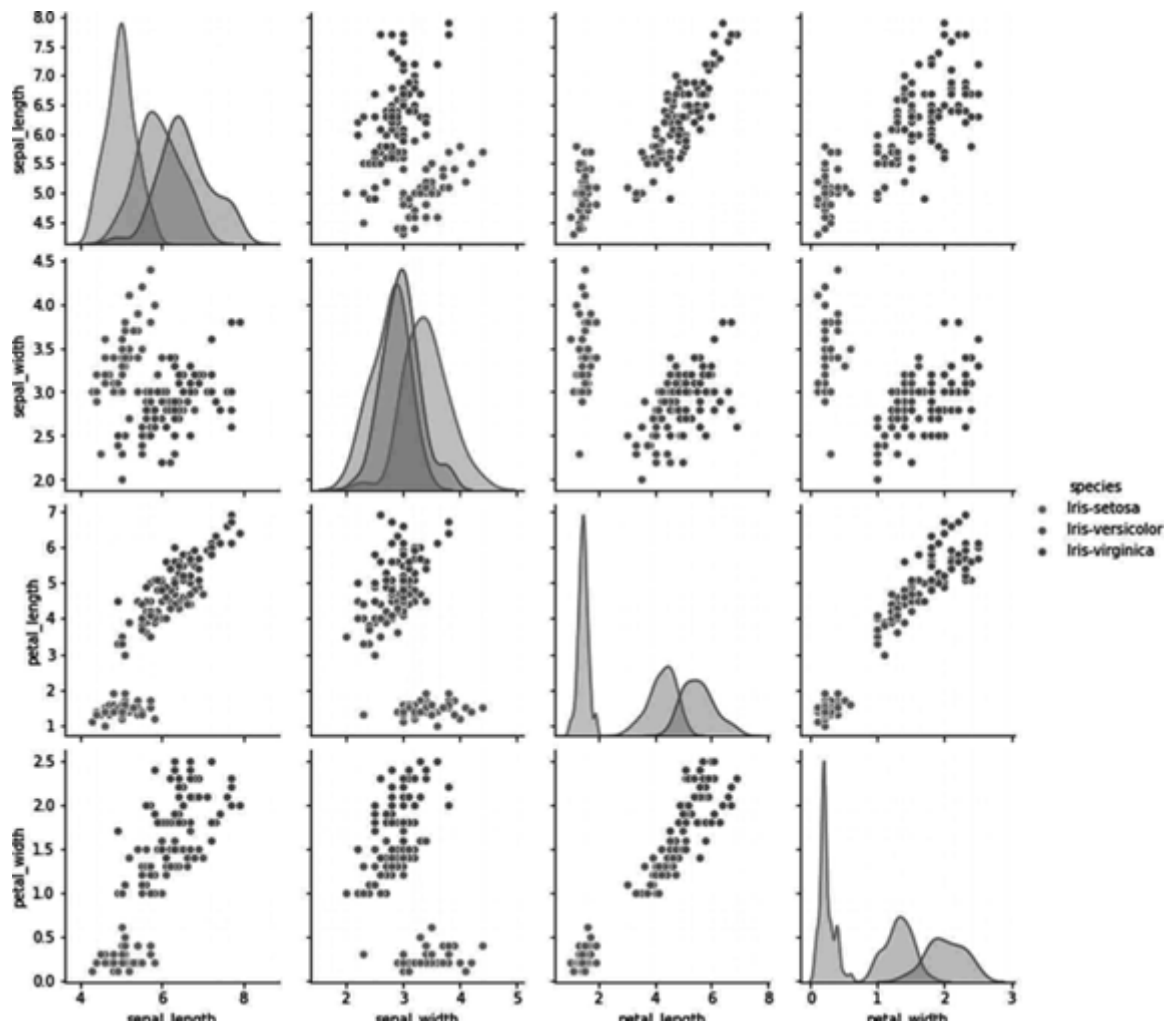


Fig. 9.9 Program output

Bước 4: Phân chia dữ liệu kiểm tra (test) và huấn luyện (train)

```
# Tách các biến độc lập khỏi các biến phụ thuộc
x = iris.iloc[:, :-1]
y = iris.iloc[:, 4]
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.30)
```

Bước 5: Tải mô hình và huấn luyện

```
model = SVC()
model.fit(x_train, y_train)
```

Đầu ra:

```
SVC(C=1.0, break_ties=False, cache_size=200,
    class_weight=None, coef0=0.0, decision_function_shape='ovr',
    degree=3, gamma='scale', kernel='rbf', max_iter=-1,
```

```
probability=False, random_state=None, shrinking=True,  
tol=0.001, verbose=False)
```

Bước 6: Kết quả dự đoán

```
pred = model.predict(x_test)  
# Nhập báo cáo phân loại và ma trận nhầm lẫn  
print(confusion_matrix(y_test, pred))
```

Đầu ra:

```
[[15 0 0]  
 [ 0 14 0]  
 [ 0 2 14]]
```

```
# tạo Báo cáo Phân loại  
print(classification_report(y_test, pred))
```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	15
Iris-versicolor	0.88	1.00	0.93	14
Iris-virginica	1.00	0.88	0.93	16
accuracy			0.96	45
macro avg	0.96	0.96	0.96	45
weighted avg	0.96	0.96	0.96	45

Naïve Bayes

Thuật toán Naive Bayes có thể được mô tả như một thuật toán Phân loại có giám sát dựa trên **định lý Bayes** với giả định **độc lập giữa các đặc trưng**. Nó cũng là một kỹ thuật phân loại.

Định lý Bayes là một nguyên tắc để xây dựng các bộ phân loại đằng sau kỹ thuật phân loại này. Các yếu tố dự đoán (predictors) được cho là độc lập. Nói một cách đơn giản, việc một đặc trưng cụ thể thuộc về một lớp không liên quan đến bất kỳ đặc trưng nào khác. Sau đây là phương trình của định lý Bayes:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

Trong đó:

- $P(A | B)$ là xác suất của giả thuyết A khi biết dữ liệu B. Đây được gọi là **xác suất hậu nghiệm (posterior probability)**.
- $P(B | A)$ là xác suất của dữ liệu B khi biết giả thuyết A là đúng.
- $P(A)$ là xác suất giả thuyết A là đúng (bất kể dữ liệu). Đây được gọi là **xác suất tiên nghiệm (prior probability)** của A.
- $P(B)$ là xác suất của dữ liệu (bất kể giả thuyết).

Triển khai

Các thư viện Python cung cấp ba loại bộ phân loại Naïve Bayes:

- **Gaussian:** Giả định rằng các đặc trưng được phân phối chuẩn (Gaussian).
- **Multinomial:** Dành cho các đếm rời rạc.
- **Bernoulli:** Hữu ích nếu các vector đặc trưng là nhị phân (ví dụ: vector 0 và 1).

Trong phần triển khai, chúng ta sẽ sử dụng bộ phân loại Gaussian Naïve Bayes và bộ dữ liệu iris. Chúng ta đã biết về bộ dữ liệu iris vì chúng ta đã nghiên cứu nó trong các thuật toán trước.

Tất nhiên, trong ví dụ này chỉ liệt kê một và hai lớp. Tuy nhiên, quy trình cũng tương tự với một vector có nhiều đặc trưng và nhóm (ngoại lệ duy nhất là việc sử dụng hàm Argmax để trả về kết quả có xác suất cao nhất).

Bước 1: Nhập tất cả thư viện

```
from sklearn import datasets
import matplotlib.pyplot as plt
import pandas as pd
# nhập các gói cần thiết
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
```

Bước 2: Tải và Phân chia bộ dữ liệu

```
# tải bộ dữ liệu iris, chia nó thành tập huấn luyện và
# tập xác thực (validation set)
iris = datasets.load_iris()
class_names = iris.target_names
iris_df = pd.DataFrame(iris.data, columns=iris.feature_names)
iris_df['target'] = iris.target
iris_df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

iris_df.describe()

Đầu ra:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

Chúng ta có 4 đặc trưng và một mục tiêu dạng danh mục vì chúng ta có 3 cấp độ (Setosa, Versicolor và Virginica, tương ứng 0, 1 và 2). Giờ chúng ta có thể xây dựng bộ phân loại của mình:

```
# Bây giờ Tách dữ liệu thành kiểm tra và huấn luyện
X_train, X_test, y_train, y_test = train_test_split(
    iris_df[['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']],
    iris_df['target'],
    random_state=0
)
```

Bước 3: Tải mô hình và biên dịch

```
NB = GaussianNB()
NB.fit(X_train, y_train)
y_predict = NB.predict(X_test)
print("Accuracy Naive Bayes: {:.2f}".format(NB.score(X_test, y_test)))
```

Đầu ra:

Accuracy Naive Bayes: 1.00

Như bạn có thể thấy, tất cả dữ liệu kiểm tra đều được thuật toán của chúng ta phân loại chính xác. Đây là một hiệu suất tốt, tốt nhất mà chúng ta từng đạt được. Nhưng hãy xem xét kích thước bộ dữ liệu yếu của chúng ta (chỉ 150 quan sát). Độ chính xác tiêu chuẩn 100% trên dữ liệu thực tế rất khó đạt được. Ngoài ra, lập luận về lỗi bằng không cũng có thể phản tác dụng. Thật vậy, chúng ta thích có một thuật toán có thể **tổng quát hóa** và không tuân thủ hoàn hảo dữ liệu kiểm tra, bởi vì mục tiêu cuối cùng của bất kỳ thuật toán ML nào là đưa ra dự đoán chính xác về dữ liệu mới chưa biết. Nguy cơ **overfitting** (quá khớp) sẽ dẫn đến một mô hình không bền vững nếu không thể sử dụng dữ liệu mới (sử dụng quá nhiều tham số để tạo ra một mô hình hoàn toàn phù hợp với dữ liệu kiểm tra).

Ưu điểm:

- Dễ hiểu.
- Có thể được huấn luyện trên các bộ dữ liệu nhỏ.

Nhược điểm:

- Đối với các đặc trưng có tần suất bằng 0, tổng khả năng (likelihood) là 0. Có nhiều hiệu chỉnh mẫu để giải quyết vấn đề này, chẳng hạn như “hiệu chỉnh Laplace”.
- Một nhược điểm khác là nó giả định rất mạnh về tính độc lập của các đặc trưng trong lớp. Những bộ dữ liệu này trong đời thực gần như khó tìm thấy.

K-Nearest Neighbors (KNN)

Nó được sử dụng để xác định các vấn đề và khắc phục chúng. Nó thường được sử dụng để giải quyết các vấn đề phân loại. Nguyên tắc chính của thuật toán này là nó lưu trữ tất cả các trường hợp hiện có và phân loại các trường hợp mới bằng cách **bỏ phiếu theo đa số (plurality voting)** của các láng giềng. Trường hợp (dữ liệu mới) sau đó được gán cho lớp phổ biến nhất trong số **K láng giềng gần nhất** của nó, theo một hàm khoảng cách. Khoảng cách có thể là Minkowski, Manhattan, Euclidean, và Hamming.

Hãy xem xét KNN như sau:

- Về mặt tính toán, KNN tốn kém hơn các thuật toán khác được sử dụng cho các bài toán phân loại.
- Cần chuẩn hóa các biến, nếu không các biến có phạm vi (range) cao hơn có thể làm sai lệch nó.
- Tại KNN, chúng ta sẽ làm việc trên một giai đoạn như giảm nhiễu (noise reduction), tiền xử lý.

Triển khai

Bước 1: Nhập thư viện


```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
```

Bước 2: Tải và trực quan hóa dữ liệu

```
names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'class']
# Liên kết tải bộ dữ liệu: https://archive.ics.uci.edu/ml/machine-learning-databases/iris/
df = pd.read_csv('C:/Users/EETRO/Desktop/Book Code/iris.data.txt', header=None,
names=names) # đường dẫn lưu tệp
df.head()
```

Đầu ra:

	sepal_length	sepal_width	petal_length	petal_width	class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
setosa = df[df['class'] == 'Iris-setosa']
versicolor = df[df['class'] == 'Iris-versicolor']
virginica = df[df['class'] == 'Iris-virginica']
```

```
plt.plot(setosa['sepal_length'], setosa['sepal_width'], 'ro', label='setosa')
plt.plot(versicolor['sepal_length'], versicolor['sepal_width'], 'bo', label='versicolor')
plt.plot(virginica['sepal_length'], virginica['sepal_width'], 'go', label='virginica')
plt.xlabel('sepal_length')
plt.ylabel('sepal_width')
plt.legend()
plt.show()
```

Đầu ra: như trong Hình 9.10

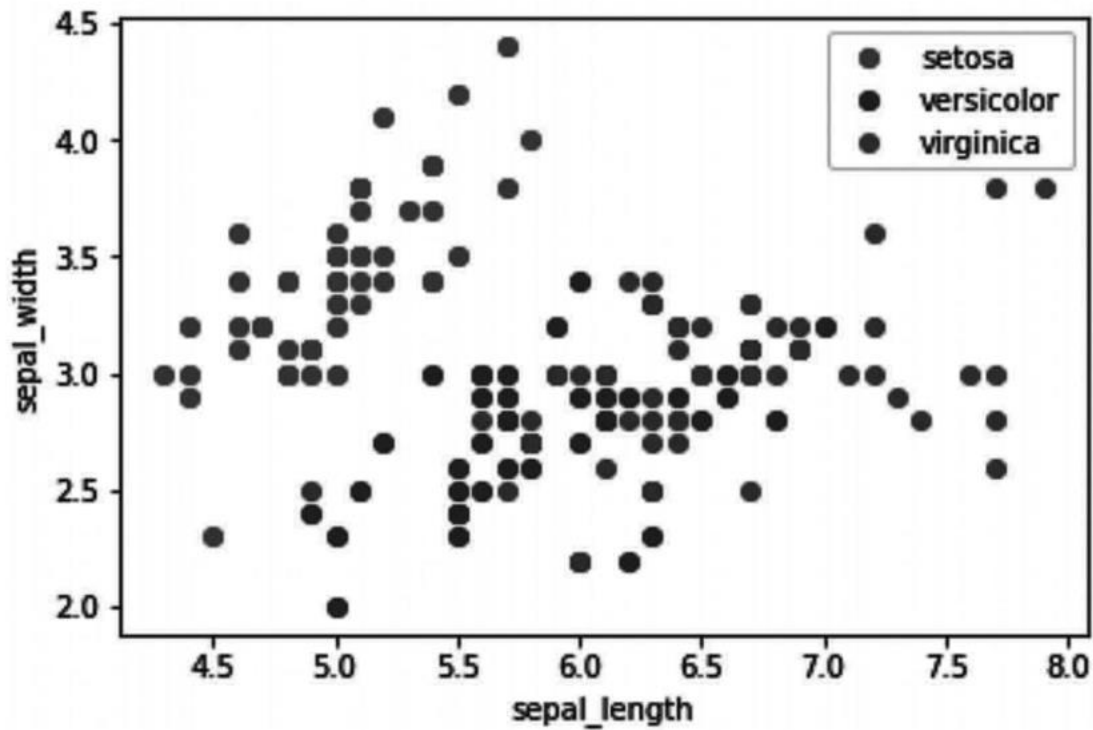


Fig. 9.10 Program Outcome

```
plt.plot(setosa['petal_length'], setosa['petal_width'], 'ro', label='setosa')
plt.plot(versicolor['petal_length'], versicolor['petal_width'], 'bo', label='versicolor')
plt.plot(virginica['petal_length'], virginica['petal_width'], 'go', label='virginica')
plt.xlabel('petal_length')
plt.ylabel('petal_width')
plt.legend()
plt.show()
```

Đầu ra: như trong Hình 9.11

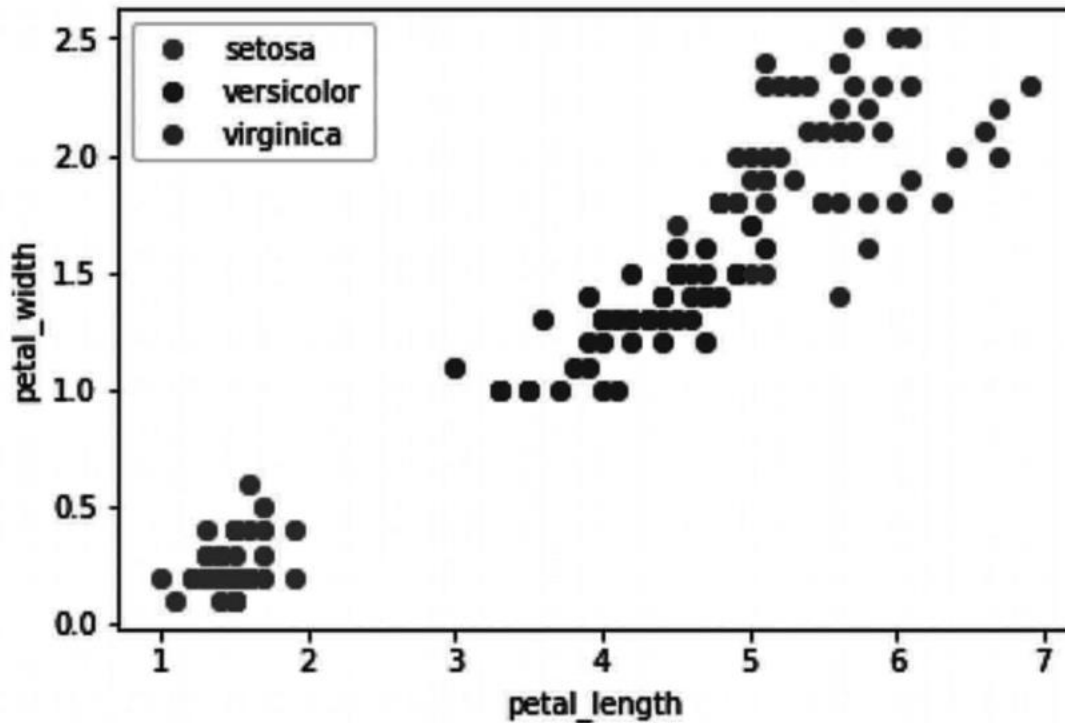


Fig. 9.11 Program outcome

Bước 3: Chia dữ liệu thành tập huấn luyện và kiểm tra với tỷ lệ 3:1

	sepal_length	sepal_width	petal_length	petal_width	class	is_train
0	5.1	3.5	1.4	0.2	Iris-setosa	True
1	4.9	3.0	1.4	0.2	Iris-setosa	True
2	4.7	3.2	1.3	0.2	Iris-setosa	False
3	4.8	3.1	1.5	0.2	Iris-setosa	True
4	5.0	3.6	1.4	0.2	Iris-setosa	True

```
df['is_train'] = np.random.uniform(0, 1, len(df)) <= .75
train, test = df[df['is_train'] == True], df[df['is_train'] == False]
print('length of train dataset: %d' % (len(train)))
print('length of test dataset: %d' % (len(test)))
```

Đầu ra:

```
length of train dataset: 111
length of test dataset: 39
```

```
train_x = train[train.columns[:len(train.columns) - 2]]
train_y = train['class']
```

```
test_x = test[test.columns[:len(test.columns) - 2]]
test_y = test['class']
```

Bước 4: Tính toán khoảng cách bằng phương pháp hàm (Euclidean Distance và Manhattan Distance)

$$EuclideanDistance = \sqrt{\sum_{i=1}^k (x_i - y_i)^2}$$

$$ManhattanDistance = \sum_{i=1}^k |x_i - y_i|$$

```
# Hàm Khoảng cách Euclidean
def euclidean_distance(data1, data2):
    distance = 0
    for i in range(data2.shape[0]):
        distance += np.square(data1[i] - data2[i])
    return np.sqrt(distance)
```

```
# Hàm Khoảng cách Manhattan
def manhattan_distance(data1, data2):
    distance = 0
    for i in range(data2.shape[0]):
        distance += abs(data2[i] - data1[i])
    return distance
```

Bước 5: Xây dựng hàm KNN

```
# Hàm KNN nhận đầu vào là
# train_x: mẫu huấn luyện,
# train_y: nhãn tương ứng,
# dis_func: hàm tính khoảng cách
# sample: một mẫu kiểm tra
# k: số lượng mẫu huấn luyện để xem xét
def knn(train_x, train_y, dis_func, sample, k):
    distances = {}
    for i in range(len(train_x)):
        d = dis_func(sample, train_x.iloc[i])
        distances[i] = d
    sorted_dist = sorted(distances.items(), key=lambda x: (x[1], x[0]))

    # lấy k láng giềng gần nhất
```

```

neighbors = []
for i in range(k):
    neighbors.append(sorted_dist[i][0])

# chuyển đổi chỉ số (indices) thành các lớp
classes = [train_y.iloc[c] for c in neighbors]

# đếm mỗi lớp trong top k
counts = Counter(classes)

# bỏ phiếu cho lớp có số lượng mẫu nhiều nhất
list_values = list(counts.values())
list_keys = list(counts.keys())
cl = list_keys[list_values.index(max(list_values))]
return cl # trả về lớp của mẫu

```

Bước 6: Lấy độ chính xác của các mô hình

```

def get_accuracy(test_x, test_y, train_x, train_y, k):
    correct = 0
    for i in range(len(test_x)):
        sample = test_x.iloc[i]
        true_label = test_y.iloc[i]
        predicted_label_euclidean = knn(train_x, train_y, euclidean_distance, sample, k)
        if predicted_label_euclidean == true_label:
            correct += 1
    accuracy_euclidean = (correct / len(test_x)) * 100

    correct = 0 # reset giá trị correct về 0
    for i in range(len(test_x)):
        sample = test_x.iloc[i]
        true_label = test_y.iloc[i]
        predicted_label_manhattan = knn(train_x, train_y, manhattan_distance, sample, k)
        if predicted_label_manhattan == true_label:
            correct += 1
    accuracy_manhattan = (correct / len(test_x)) * 100

    print("model accuracy with euclidean is %0.2f" % (accuracy_euclidean))
    print("model accuracy with manhattan is %0.2f" % (accuracy_manhattan))

```

```

get_accuracy(test_x, test_y, train_x, train_y, 5)

```

Đầu ra:

model accuracy with euclidean is 94.87
model accuracy with manhattan is 94.87

Độ chính xác là tỷ lệ giữa số lượng mẫu được xác định chính xác và tổng số lượng mẫu. Ở đây, chúng ta có thể thấy rằng việc sử dụng khoảng cách Euclidean và Manhattan làm thước đo độ tương tự đều cho độ chính xác 94.87%.

Ưu điểm:

- Không có giả định về dữ liệu.
- Thuật toán đơn giản để hiểu.
- Có thể được sử dụng cho hồi quy và phân loại.

Nhược điểm:

- Yêu cầu bộ nhớ cao - Tất cả dữ liệu huấn luyện phải nằm trong bộ nhớ để đo lường K láng giềng gần nhất.
- Nhạy cảm với các đặc trưng liên quan.
- Nhạy cảm với kích thước của dữ liệu vì phải tính toán khoảng cách đến K điểm gần nhất.

Phân cụm K-Means (K-means Clustering)

Đúng như tên gọi, nó được sử dụng để giải quyết vấn đề phân cụm. Đây là một loại **Học không giám sát**. Nguyên tắc chính của thuật toán K-Means là gán bộ dữ liệu vào các cụm khác nhau. Thuật toán phân cụm K-means được sử dụng để xác định và tìm ra các mẫu, đồng thời đưa ra quyết định tốt hơn cho các nhóm không được gán nhãn rõ ràng trong dữ liệu. Dữ liệu mới có thể dễ dàng được phân bổ vào nhóm thích hợp nhất sau khi thuật toán được thực hiện và các nhóm được xác định.

Để bắt đầu với k-means, bạn phải khởi tạo các **tâm cụm (K centroids)** một cách ngẫu nhiên. K-means là một thuật toán lặp, gồm hai bước (như trong Hình 9.12):

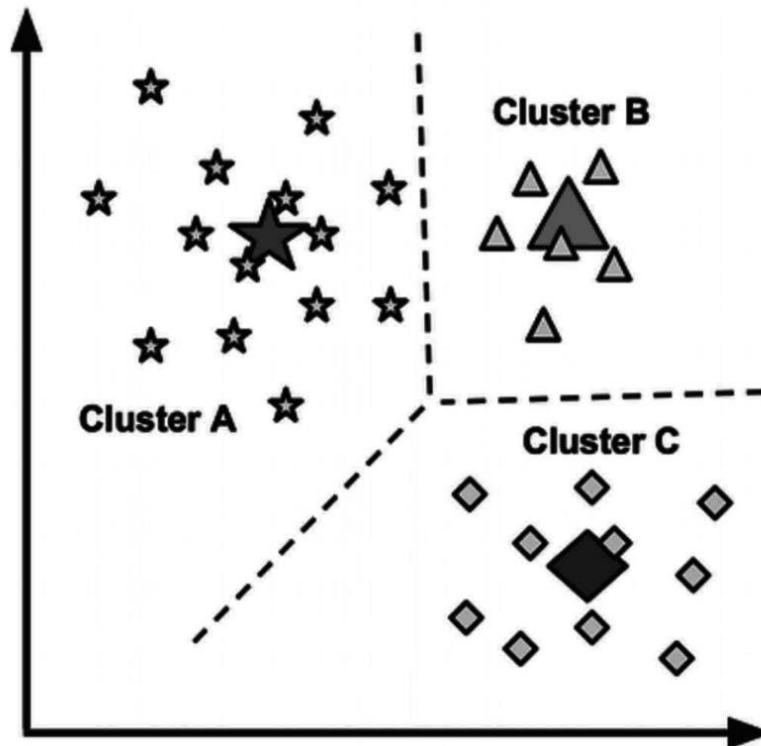


Fig. 9.12 Data Points Clustering

1. **Gán cụm (Cluster Assignment):** Thuật toán duyệt qua mọi điểm dữ liệu và gán điểm dữ liệu đó cho một trong các tâm cụm, tùy thuộc vào cụm nào gần hơn.
2. **Di chuyển tâm cụm (Move to centroid):** Các tâm cụm được di chuyển đến giá trị trung bình của các cụm được hình thành sau khi gán cụm.

Và sau đó nó lặp lại chu kỳ. Các tâm cụm không thể di chuyển nữa sau một số bước nhất định và sau đó các lần lặp có thể kết thúc.

Triển khai

Bước 1: Nhập thư viện

```
from sklearn import datasets
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.cluster import KMeans
```

Bước 2: Tải dữ liệu

```
iris = datasets.load_iris()
```

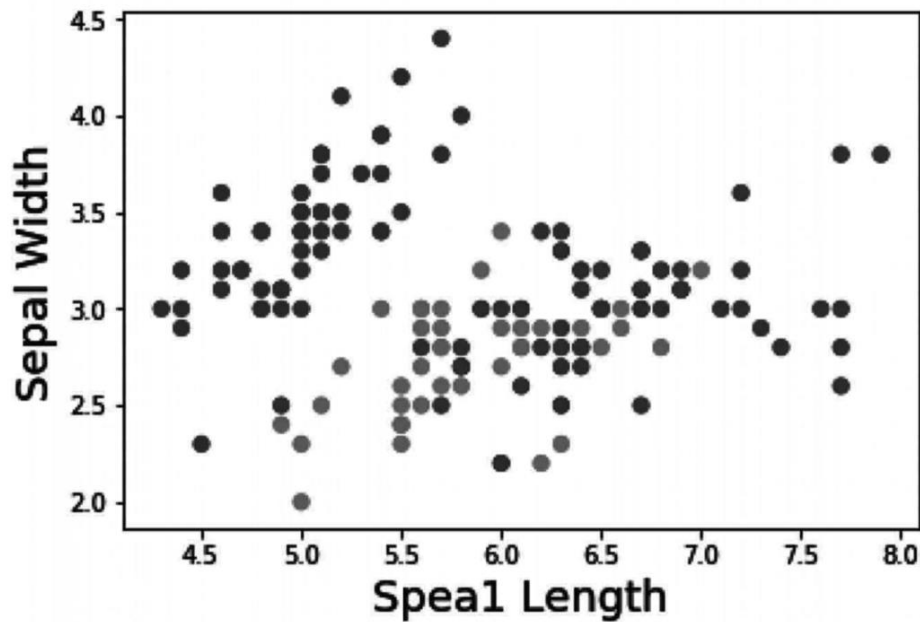
Bước 3: Xác định mục tiêu và các yếu tố dự đoán (Predictors)

```
X = iris.data[:, :2]  
y = iris.target
```

Bước 4: Trực quan hóa dữ liệu

```
plt.scatter(X[:,0], X[:,1], c=y, cmap='gist_rainbow')  
plt.xlabel('Sepal Length', fontsize=18)  
plt.ylabel('Sepal Width', fontsize=18)
```

Đầu ra: như trong Hình 9.13



Hình 9.13 Program outcome

Bước 5: Tải và Biên dịch mô hình

```
km = KMeans(n_clusters=3, n_jobs=4, random_state=21)  
km.fit(X)
```

Đầu ra:

```
KMeans(algorithm='auto', copy_x=True, init='k-means++',  
       max_iter=300, n_clusters=3, n_init=10, n_jobs=4,  
       precompute_distances='auto', random_state=21, tol=0.0001,  
       verbose=0)
```

Bước 6: Lấy Tâm cụm (Centroid)

```
centers = km.cluster_centers_  
print(centers)
```


Đầu ra:

```
[[5.77358491 2.69245283]  
 [5.006    3.428    ]  
 [6.81276596 3.07446809]]
```

Bước 7: So sánh dữ liệu gốc và dữ liệu đã phân cụm

điều này sẽ cho chúng ta biết các quan sát dữ liệu thuộc về cụm nào.

```
new_labels = km.labels_
```

Vẽ các cụm đã xác định và so sánh với câu trả lời

```
fig, axes = plt.subplots(1, 2, figsize=(16,8))
```

```
axes[0].scatter(X[:, 0], X[:, 1], c=y, cmap='gist_rainbow', edgecolor='k', s=150)
```

```
axes[1].scatter(X[:, 0], X[:, 1], c=new_labels, cmap='jet', edgecolor='k', s=150)
```

```
axes[0].set_xlabel('Sepal length', fontsize=18)
```

```
axes[0].set_ylabel('Sepal width', fontsize=18)
```

```
axes[1].set_xlabel('Sepal length', fontsize=18)
```

```
axes[1].set_ylabel('Sepal width', fontsize=18)
```

```
axes[0].tick_params(direction='in', length=10, width=5, colors='k', labels=20)
```

```
axes[1].tick_params(direction='in', length=10, width=5, colors='k', labels=20)
```

```
axes[0].set_title('Actual', fontsize=18)
```

```
axes[1].set_title('Predicted', fontsize=18)
```

Đầu ra: như trong Hình 9.14

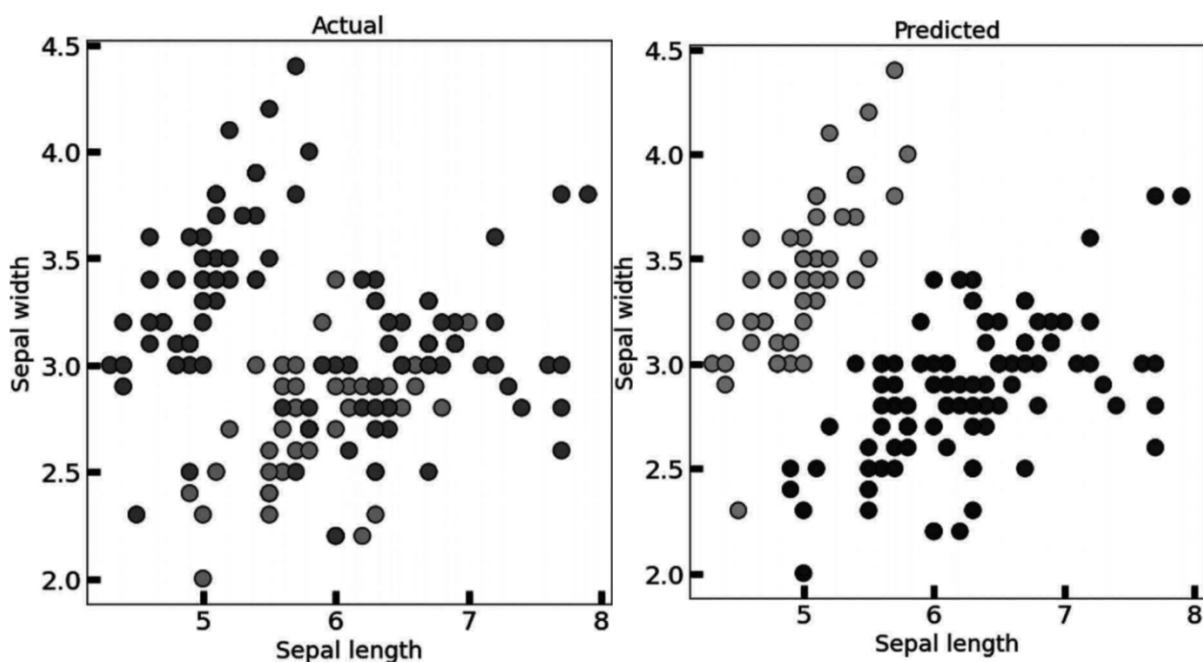


Fig. 9.14 Program outcome

Ưu điểm:

- K-means đơn giản và hiệu quả về mặt máy tính.
- Nó rất trực quan và nhanh chóng để hình dung các ảnh hưởng.

Nhược điểm:

- K-means phụ thuộc nhiều vào quy mô (scale) và không được thiết kế cho dữ liệu có mật độ và hình dạng khác nhau.
- Việc ước tính hiệu suất chủ quan hơn. Nó cần nhiều đánh giá của con người hơn là các chỉ số tin cậy.

Random Forest (Rừng ngẫu nhiên)

Đây là một thuật toán phân loại có giám sát. Thuật toán random forest có ưu điểm là có thể được sử dụng cho các bài toán phân loại cũng như hồi quy.

Về cơ bản, đây là tập hợp các cây quyết định (khu rừng) hoặc một **tập hợp (ensemble)** của các cây quyết định.

Nguyên tắc cơ bản của random forest là mỗi cây đều được “chấm điểm” (graded) và “khu rừng” sẽ chọn ra những “điểm” tốt nhất (lấy kết quả bỏ phiếu đa số).

Sự khác biệt giữa thuật toán random forest và cây quyết định là quá trình xác định nút gốc và phân chia các nút chức năng diễn ra ngẫu nhiên trong random forest.

Triển khai

Bước 1: Nhập tất cả các thư viện cần thiết

```
# Nhập Mô hình Random Forest
from sklearn.ensemble import RandomForestClassifier
# Nhập thư viện dataset của scikit-learn
from sklearn.model_selection import train_test_split
from sklearn import datasets
import pandas as pd
# Nhập mô-đun metrics của scikit-learn để tính độ chính xác
from sklearn import metrics
```

Bước 2: Tải và Trực quan hóa dữ liệu

```
# Tải bộ dữ liệu
iris = datasets.load_iris()
# in tên các loài (setosa, versicolor, virginica)
print(iris.target_names)
# in tên của bốn đặc trưng
print(iris.feature_names)
```

Đầu ra:

```
['setosa' 'versicolor' 'virginica']
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
```

```
# in dữ liệu iris (5 bản ghi đầu tiên)
print(iris.data[0:5])
# in nhãn iris (0:setosa, 1:versicolor, 2:virginica)
print(iris.target)
```

Đầu ra:

[illegible]

```
# Tao một DataFrame từ bộ dữ liệu iris đã cho.
```

```
data = pd.DataFrame({
    'sepal length': iris.data[:,0],
    'sepal width': iris.data[:,1],
    'petal length': iris.data[:,2],
    'petal width': iris.data[:,3],
    'species': iris.target
})
data.head()
```

Đầu ra:

	sepal length	sepal width	petal length	petal width	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Bước 3: Phân chia dữ liệu huấn luyện và kiểm tra

```
# Nhập hàm train_test_split
X = data[['sepal length', 'sepal width', 'petal length', 'petal width']] # Đặc trưng
y = data['species'] # Nhãn
# Chia bộ dữ liệu thành tập huấn luyện và tập kiểm tra
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3) # 70% huấn luyện
và 30% kiểm tra
```

Bước 4: Nhập mô hình và biên dịch

```
# Tạo một bộ phân loại Gaussian (ở đây là Random Forest)
clf = RandomForestClassifier(n_estimators=100)
# Huấn luyện mô hình bằng cách sử dụng các tập huấn luyện
clf.fit(X_train, y_train)
```

Bước 5: Xem độ chính xác và dự đoán của mô hình

```
y_pred = clf.predict(X_test)
# Độ chính xác của mô hình, bộ phân loại đúng bao nhiêu lần?
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
```

Đầu ra: (Lưu ý: Độ chính xác có thể thay đổi do train_test_split ngẫu nhiên, văn bản gốc ra 1.0)

Accuracy: 1.0

```
# Dự đoán một mẫu mới
prediction = clf.predict([[3, 5, 4, 2]])
species_idx = prediction[0]
print(iris.target_names[species_idx])
```

Đầu ra:

'versicolor'

Ưu điểm:

- Bộ phân loại random forest có thể được sử dụng cho các chức năng phân loại và hồi quy.
- Có thể quản lý các giá trị bị thiếu.
- Ngay cả khi chúng ta có nhiều cây hơn trong rừng, điều này sẽ không làm mô hình bị overfit (quá khớp).

Nhược điểm:

- Về bản chất, một mô hình ensemble (tổ hợp) ít diễn giải hơn một cây quyết định.
- Một số lượng lớn các cây sâu có thể tốn kém chi phí (nhưng có thể chạy song song) và sử dụng nhiều bộ nhớ.
- Dự đoán chậm hơn, điều này có thể gây ra các vấn đề khi triển khai.

Tóm tắt

Các thuật toán học máy đang được sử dụng cho rất nhiều mục đích và giúp chúng ta có được kết quả hữu ích bằng cách phân tích dữ liệu có sẵn từ một số tài nguyên. Ở đây chúng ta có các thuật toán khác nhau của học máy như hồi quy tuyến tính, hồi quy logistic, cây quyết định, v.v., đã được thảo luận trong chương này với việc triển khai bằng cách sử dụng bộ dữ liệu thời gian thực tên là Bộ dữ liệu Iris, đây là bộ dữ liệu nổi tiếng nhất bao gồm dữ liệu về các loài hoa. Từ phần triển khai của thuật toán, chúng ta có thể hiểu cách các thuật toán này có thể được triển khai trong thời gian thực. Ưu điểm và nhược điểm của các thuật toán cũng đã được nêu để hiểu rõ hơn.